

React for This Project

A from-scratch introduction to React — assuming no prior exposure to it at all, only HTML/CSS/JavaScript — built entirely around how this codebase actually uses it.

The one-sentence version: instead of writing code that says *"find this element and change it"* (the way you'd write plain JavaScript/DOM code), you write a function that says *"here is what the screen should look like right now, given the current data"* — and React takes care of comparing that to what's currently on screen and updating only what actually changed. Everything else in this document is really just detail on top of that one idea.

Contents

1. The problem React solves – imperative vs. declarative
2. JSX – HTML-looking syntax inside JavaScript
3. Components – a function that returns UI
4. Props – passing data in, including `children`
5. `useState` – giving a component memory – the most-used hook here (55×)
6. What "re-render" actually means – what happens after a state update
7. Handling events – `onClick`, `onChange`
8. Conditional rendering – the three patterns used in this codebase
9. Rendering lists with `.map()` and `key` – 88 uses across the app
10. `useEffect` – reaching outside React – side effects, cleanup, dependencies
11. `useMemo` and `useRef` – the two less-common hooks used here
12. Custom hooks – packaging reusable stateful logic
13. Where global state actually lives – no Context, Zustand instead
14. Glossary – quick reference

1. The problem React solves

In plain JavaScript, updating the page is something *you* do, one instruction at a time — find a node, change a property:

```
document.getElementById('streak').textContent = String(streak);
document.getElementById('streak-dot').style.background = goalMet ? 'var(--acc)' : 'var(--line2)';
```

This works fine for a couple of elements. It gets painful once a screen has dozens of things that can each change for different reasons (a due-card badge, a theme, a progress bar, a hover tooltip) — you end up having to remember, by hand, every single place in your code that needs to run again whenever any one piece of data changes. Miss one, and the screen quietly shows stale information.

React's answer: you write one function per piece of UI that **describes the whole thing from scratch every time**, based only on the current data — and React figures out, behind the scenes, exactly which real DOM nodes need to change to match. You never call `document.getElementById` yourself; you just say what the result should look like, and re-say it (mentally) every time something changes.

2. JSX — HTML-looking syntax inside JavaScript

A React component's "here's what it should look like" description is written in **JSX** — not a new language, just JavaScript with the ability to write tags directly:

```
function Greeting() {
  const name = 'Rafsan';
  return (
    <div className="greeting">
      Hello, {name}!
    </div>
  );
}
```

`{'←'}` curly braces drop back into plain JavaScript

Under the hood, that JSX is converted (by the build tools from the frontend document) into plain JavaScript function calls that build up a description of the UI — you never see that generated code, but it's why JSX can live inside a `.tsx` file and still just be "JavaScript" as far as the language is concerned. A few concrete differences from plain HTML you'll notice immediately in this codebase:

HTML	JSX	WHY
<code>class="..."</code>	<code>className="..."</code>	<code>class</code> is a reserved word in JavaScript (used for defining classes), so JSX renamed the attribute.
<code>onclick="..."</code>	<code>onClick={'{'}fn{'}'}</code>	Event handlers are camelCase and take an actual JavaScript function value, not a string of code.
<code></code>	<code></code>	Every tag must be explicitly closed or self-closed — JSX has no HTML-style "void elements" exception.
—	<code>{'{' }expression{'}'}</code>	Anything inside single curly braces is evaluated as a normal JavaScript expression and inserted as text/UI.

3. Components — a function that returns UI

A **component** is just a JavaScript function whose return value is JSX. Once defined, it can be used like a custom HTML tag anywhere else in the app. This project has one top-level component per screen (`Dashboard()`, `Review()`, `Forest()`, ...), each usually built from smaller components defined in the same file. A real, minimal example from

`renderer/src/features/dashboard/Dashboard.tsx`:

```
function CardLabel({ children }) {
  return (
    <div className="font-mono text-[9px] uppercase tracking-[0.12em] text-tx3">
      {children}
    </div>
  );
}

// used elsewhere in the same file, exactly like a built-in tag:
<CardLabel>Due now</CardLabel>
```

Component names are always capitalized (`CardLabel`, not `cardLabel`) — this isn't just style, it's how JSX itself tells a custom component apart from a plain HTML tag like `<div>`.

4. Props — passing data in

Data flows into a component the same way arguments flow into a function — written as attributes on the tag, received as a single object parameter, usually destructured immediately:

```
function Card({ children, className = '' }) {
  return <div className={`rounded-panel border border-line2 bg-panel p-[18px] shadow-panel ${className}`}>
    {children}
  </div>;
}

// caller decides the content and an extra class:
<Card className="flex flex-col gap-[14px]">
  <CardLabel>Due now</CardLabel>
  { /* ...more JSX here... */ }
</Card>
```

`children` is a special prop: it's whatever was written *between* a component's opening and closing tags, rather than passed as a named attribute. This is exactly how you'd wrap arbitrary HTML inside a reusable container in plain markup — `Card` doesn't need to know what's inside it, only where to put it.

Props flow one way: parent → child. A component can never reach back up and change the props it was given — if a child needs to change something the parent controls, the parent must pass down a *function* as a prop (an event-handler prop) for the child to call. Section 7 shows this in practice.

5. `useState` — giving a component memory

A plain variable inside a function is forgotten the moment the function returns — exactly the problem a component has, since React calls it again and again over the component's lifetime.

`useState` is the fix: it gives a component a value that **survives between calls**, plus a function to update it that also tells React *"re-run this component, something changed."* This is the single most-used hook in the project — 55 occurrences across the renderer. From

`renderer/src/features/player/Player.tsx`:

```
const [toast, setToast] = useState(''); // current value: '', updater: setToast

// later, in response to a click:
const handleSave = () => {
  setSaved(true);
  setToast(`clip extraction queued · ${word} · 6.4 s`); // triggers a re-render with the new
  text
};
```

`useState('')` returns a pair: the *current* value (here, starting as an empty string) and a function that replaces it. Calling `setToast(...)` never mutates `toast` directly — it schedules a fresh call to the component function, and *that* call will see the new value. This is why you'll never see `toast = '...'` anywhere in this codebase — reassigning the variable directly wouldn't tell React anything changed.

6. What "re-render" actually means

- 1 Something calls a state setter — a click handler, an effect, anything. React marks that component as needing to run again.
- 2 React calls the component function from the top, as an ordinary function call. All the JSX in its `return` is recomputed using the *new* state value.
- 3 React compares this new JSX description to what was there before (a process usually called "reconciliation" / the "virtual DOM diff") and works out the minimal set of real, actual DOM changes needed.
- 4 Only those specific DOM nodes are touched — the rest of the page is left completely alone, even though the whole function "ran again" in step 2.

The practical upshot: it's normal and expected for a component function to run often. What's expensive is real DOM manipulation, and React's whole job is minimizing that — you get to

write simple, re-run-everything logic while React handles the efficiency underneath.

7. Handling events

Event handler props (`onClick`, `onChange`, ...) take a plain function. This project mixes two styles depending on complexity — a short inline arrow function for one-liners, and a separately-named function when there's real logic, both from `Review.tsx`:

```
<button onClick={() => setRevealed(true)}>Show answer</button>    // inline – trivial, one call

// named – enough logic to deserve a name and be reused across two buttons:
const handleRate = () => {
  if (index + 1 >= total) setSessionDone(true);
  else { setIndex((i) => i + 1); setRevealed(false); }
};
// ...
<button onClick={handleRate}>{r.label}</button>
```

Note `onClick={handleRate}`, not `onClick={handleRate()}` — the prop needs the function itself (to be called later, on click), not the result of calling it immediately while rendering. This is a very common first mistake, and worth remembering explicitly.

8. Conditional rendering

JSX has no built-in `if` tag — you reach for plain JavaScript. Three patterns cover every case in this codebase:

PATTERN	REAL EXAMPLE	WHEN IT'S USED HERE
Early <code>return</code>	<pre>if (sessionDone) return (<DoneScreen />); return (<NormalScreen />);</pre>	<code>Review.tsx</code> — two entirely different screens depending on a boolean. Clearest option when the two outcomes barely share any markup.
<code>&&</code> short-circuit	<pre>{'{' }hintOn && (<div>... </div>){'{' }</pre>	<code>ConversationLive.tsx</code> — show this element, or nothing at all. If <code>hintOn</code> is false, JSX simply renders nothing for that expression.
Ternary <code>? :</code>	<pre>{'{' }micState === 'recording' ? 'Stop' : 'Start'{'{' }</pre>	<code>Challenge.tsx</code> — pick between exactly two small values/elements inline, inside otherwise-shared markup.

9. Rendering lists with `.map()` and `key`

To turn an array of data into an array of elements, this project uses plain JavaScript's `.map()` — 88 times across the renderer. From `renderer/src/features/review/Review.tsx`:

```
{RATINGS.map((r, i) => (
  <button key={r.label} onClick={handleRate} style={{ borderColor: RATING_STYLE[i].bd }}>
    {r.label}
  </button>
))}
```

Why `key` is not optional. When an array of elements can change (items added, removed, or reordered), React needs a stable way to tell "this is the same button as before, just moved" apart from "this is a brand-new button." Without `key`, React falls back to matching by position, which can silently mix up which internal state belongs to which item after a reorder. The value must be a string/number that's stable and unique among siblings — here, `r.label` (e.g. `"Again"`) works because rating labels never repeat or reorder within one list.

10. `useEffect` — reaching outside React

Everything so far describes UI as a function of data — but some things aren't UI at all: starting a timer, subscribing to a browser event, calling an API when a screen first opens. `useEffect` is the hook for exactly that category of code — it runs *after* React has updated the real DOM, separate from the render itself. Used 20 times in this codebase. The clearest example, `Player.tsx`'s toast auto-dismiss:

```
useEffect(() => {
  if (!toast) return;
  const t = setTimeout(() => setToast(''), 2800); // the actual side effect: a browser timer
  return () => clearTimeout(t); // cleanup – cancels the old timer
}, [toast]); // dependency array
```

- 1 The **dependency array** (`[toast]`) tells React "only re-run this effect when `toast` changes" — not on every unrelated re-render of the component.
- 2 Each time it does run, it starts a new 2.8-second timer that will clear the toast text.
- 3 The function *returned from* the effect is a **cleanup function** — React calls it right before running the effect again (or when the component disappears entirely), which cancels the previous timer so two overlapping toasts can't both fire.

An empty dependency array, `[]`, means "run once, right after this component first appears, and never again" — the standard "on mount" pattern, used in `App.tsx` to kick off the whole app:

```
useEffect(() => { void initialize(); }, [initialize]);
// `initialize` is a stable function reference, so in practice this runs once.
```

11. `useMemo` and `useRef` — the two less-common hooks used here

11.1 `useMemo` — skip recalculating when nothing relevant changed

```
// renderer/src/features/forest/Forest.tsx
const tiles = useMemo(() => buildTiles(biome), [biome]);
```

`buildTiles` generates an 81-tile isometric grid using a seeded random-number sequence (a real function, not free). Without `useMemo`, that entire grid would be recomputed from scratch on every re-render of the Forest screen — including ones with nothing to do with the grid at all, like the mouse moving over a hover tooltip. `useMemo` caches the result and only recalculates it when `biome` itself changes.

11.2 `useRef` — a value that persists but doesn't trigger a re-render

```
// renderer/src/features/reader/Reader.tsx
const scrollRef = useRef<HTMLDivElement>(null);
// ...
<div ref={scrollRef} className="overflow-y-auto">...</div>
// ...later:
scrollRef.current?.scrollTo({ top: 0 }); // direct access to the real DOM element
```

A ref is like a small mutable box (`{'current': ...}`) that keeps its value between renders — the key difference from `useState` is that **changing a ref never causes a re-render**. Two situations call for it: getting a handle to an actual DOM element (as above, to call `.scrollTo()` directly — something no amount of JSX describing the UI can do, since scrolling isn't part of "what does the screen look like"), and tracking a value across renders that should never itself cause the screen to redraw (a plain flag, a timestamp).

12. Custom hooks — packaging reusable stateful logic

A **custom hook** is just a regular function — by convention named starting with `use` — that itself calls other hooks. It's the tool for reusing "component-shaped" logic across screens, without copy-pasting the same `useEffect` / `useRef` / `useState` combination everywhere. This project's one custom hook, `renderer/src/features/reader/useReadingSession.ts` (simplified):

```
export function useReadingSession(bookId: string | null): void {
  const heartbeat = useReaderStore((s) => s.heartbeat); // a hook, called inside another
hook
  const lastBeatRef = useRef(Date.now());

  useEffect(() => {
    if (!bookId) return;
    const timer = setInterval(() => { /* send a heartbeat */ }, 30_000);
    return () => clearInterval(timer);
  }, [bookId]);
}

// used in Reader.tsx with a single line – the caller doesn't need to know
// any of the above exists:
useReadingSession(bookId);
```

All the timer/cleanup/activity-tracking complexity for "record how long the user has been reading" lives in exactly one place, and `Reader.tsx` itself stays focused on what it actually renders.

13. Where global state actually lives

Many React tutorials introduce **Context** (`createContext` / `useContext`) as the way to share data across distant components without passing props down through every layer in between. This codebase uses **zero** Context calls. Instead, anything that needs to be read or changed from many unrelated components — which screen is showing, the dark/light theme, the signed-in user, the entire onboarding wizard's data — lives in a **Zustand store** (covered fully in the companion frontend document), a small library built specifically for this job:

```
const theme = useShellStore((s) => s.theme); // any component, anywhere, no Provider wrapper
needed
```

Practically speaking: if you're looking for "where does this piece of data come from" and it isn't a prop passed from a parent and isn't local `useState`, it's almost certainly a field on one of the three stores under `renderer/src/store/` (`appStore`, `shellStore`, `onboardingStore`) — not React Context.

14. Glossary — quick reference

TERM	ONE-LINE DEFINITION
component	A function that returns JSX — usable like a custom HTML tag.
JSX	HTML-looking syntax written directly inside JavaScript/TypeScript.
props	Data passed into a component from its parent, read-only, one-way.
children	The special prop holding whatever was written between a component's opening/closing tags.
state	A value that persists across a component's re-renders and triggers a re-render when changed via its setter.
useState	The hook that creates one piece of component state and its updater function.
re-render	React calling a component function again to get its latest JSX, then updating only the real DOM nodes that actually changed.
useEffect	Runs side-effect code (timers, subscriptions, API calls) after a render, optionally cleaned up before the next one.
dependency array	The list telling an effect (or <code>useMemo</code>) which values it should re-run for.
cleanup function	The function an effect can return, run before the effect re-runs or the component disappears.
useMemo	Caches an expensive calculation, recomputing only when its dependencies change.
useRef	A mutable value that survives across renders without ever causing one itself; also used to reference a real DOM element.
custom hook	A function starting with <code>use</code> that calls other hooks, packaging reusable stateful logic.
key	The stable, unique identifier React needs on each item of a rendered list to track it correctly across changes.
Context	React's built-in cross-component sharing mechanism — not used in this project; Zustand stores fill this role instead.

Companion reading: *FluencyOS — Frontend, Electron & Onboarding Explained, Tailwind CSS for This Project, and TypeScript for This Project* cover the rest of the frontend stack the same way this document covers React. Every example above is taken directly from the current `renderer/src` source, with file paths given so each one can be opened and shown live during a demo.